

## Testing In SDLC

### Software Development & Software Testing:

**What is SDLC Model or Process Model:** It is a structured blueprint that explains how software gets built from idea to delivery and how testing fits into that journey.

It defines activities in each phase (what should be done, what results should you wait for, what production should you see).

It shows logical relationship between phases (what depends on what).

Describes the iteration mode (waterfall, agile, spiral).

We have different SDLC models each defines those in different way. Those 2 are sequential, iterative and incremental.

### V-Model Sequential:

Although the V-Model is sequential, backtracking to earlier phases is possible if serious defects are discovered, but it is expensive and not done repeatedly.

in the V SDLC, we plan the tests at the same time we do the phases. for an example, we plan the acceptance tests once the requirement specifications start. but we actually execute the tests when we finish programming the entire project. and the tests go down to up.

### **Phases:**

**Requirements Definition -----→ Acceptance Test**

**Functional System Design -----→ System Test**

**Technical System Design -----→ Integration Test**

**Components Specification -----→ Component Test**

**Programming**

**Requirements Definition:** create functional and non-functional requirements, solve everything related to specifications with stakeholders, create the system requirement specifications, create user stories from the user or the client.

**e.g.:** The system shall allow users to reset passwords via email.

**Functional System Design:** Design the workflow, Design the UI prototype, design the system logic and how the flow works from a business/user perspective, create use case diagrams. Create the dialog process.

**e.g.:** A workflow diagram for online shopping checkout (cart → address → payment → confirmation).

**Technical System Design:** define how the system will be built technically, the design architecture the database scheme and all the other technical stuff. Design the system's technical implementation.

**e.g.:** Selecting React for frontend, Node.js for backend, and MongoDB for database.

**Components Specification:** define each module, class, component, create class diagrams. Designing the internal structure, the interfaces.

**e.g.:** Specification for a Payment Module: here it goes for example.

**Acceptance Test:** Validates the user needs, business needs, and legal compliance of the final product. Confirms the system is ready for real users.

**e.g.:** Students test a new learning app to confirm it's easy to use and meets their study needs.

**Q: Does the system exhibit agreed performance characteristics from the customer's point of view?**

**System Test:** test the entire system, done in an entire environment. Performed by testers not developers. It verifies the system against the system requirements.

**e.g.:** Security testing an admin panel → ensure non-admins cannot access restricted pages.

**Q: Does the system as a whole fulfill the specified requirements?**

**Integration Test, Unit Integration Test:** test the interactions between multiple components, done by developers.

**e.g.:** Frontend form + backend API → ensure submitted student data reaches the server correctly.

**Q: Do components interact as specified in the technical system design?**

**Component Test:** tests each component: class, module. Done by developers, Focuses on: logic correctness, data handling, error cases. It's by far the lowest level of testing.

**e.g.:** Testing a function that calculates student GPA → input grades, verify correct average output.

**Q:** Does every software or hardware component fulfill its specification?

### **Properties of V-SDLC:**

Sequential Flow with Testing Mapping: Tests are planned early and executed later, opposite the development flow making the V shape.

Emphasis on Verification and Validation: Verification is done during development (left side) Validation is done during testing (right side)

Early Test Planning: Test planning/design occurs alongside the corresponding development phase.

Strict Phase Structure: Phases must finish before the next one starts, making the model predictable but less flexible.

**Constructive Activities:** The constructive activities are those on the left side of the V shape, the planning itself

**Analytical Activities:** Those are the tests, those are on the right side of the V shape

**Motivation:** Errors are easier and more efficient to be found on the same level of abstraction. To be more specific. It does not make any sense to use an integration testing to validate a requirement, or how can you test a requirement to the acceptance criteria while the component testing is not done or you have a problem with a function!!!!

**An additional test: System integration test:** tests the interaction between different systems after the system was tested in the environment. For instance we have an app for bank, after we test our app, we can conduct a test to test the interaction between the app and the ATM (another system)

**Verification:** it takes place on the left side (constructive) of the V-model. It helps verify whether we built it right according to the requirements that we have. It takes place at each level on the left side.

Verification is phase-specific: You verify what was defined in that phase. You do not wait until coding is complete to verify requirements or design.

Since it happens at each phase, this ensures that each phase is correct and complete.

Goal is to answer whether each phase was done correctly or not ( according to our requirements, we don't care about the user here)

**Validation:** each level validates the output against the requirements from a user stand point, this test answers whether the right feature or component was done.

**When does testing start when using the V-Model?:** test planning starts directly with each crossoponding constructive phase, while test execution actually starts when the programming is finished in a reversed way of the planning giving us the V shape.

### **Iterative-Incremental Process Models**

In this software development model we build the system piece by piece, we don't make one (full) delivery of the system, but rather features each increment (sprint) that works correctly and validated (tested).

Each increment we add a new feature, we fix bug in an old feature, and we improve the overall quality.

**Purely Iterative:** this model is used for refining the features of an already existing full app, through defined iterations the app's quality gets better. We already know the main goals, ideas of the working app.

**Purely Incremental:** this model is used for building the app the system goals are figured out and added gradually through increments. You can expand or decrease the scope at any time. Increments have fixed duration, during which we add features.

### **Testing in Iterative-Incremental Development Models:**

- We have tiny W model per iteration where we develop → test → develop (fix) → test. We don't apply this to the entire project ones, but we do the same for each iteration
- Test phases and test stages overlap often; because work is fast and cyclic different test often takes places at the same time, you may be conducting unit testing while other developer conduction integration testing at the same time.
- Test each feature on different testing levels if possible; integration, component, with the entire system, UAT.
- Use reusable tests if possible, each iteration or feature you should reuse the same tests.
- When adding new feature or component you should add specific tests for it, and use these reusable tests on it as well.
- You should conduct integration testing in addition to regression testing. Integration testing is to make sure all the components work together, while the regressing testing is to make sure that the old tested features are still working correctly after the new features have been added.
- Each increment, we should validate and verify.
- Use CI/CD pipelines, many test are redundant so we should aim to use automated ones

#### **Tips for high quality Tests:**

- For each thing you develop you should create a test for it, testing and developing should be coupled.
- Start testing as early as possible don't wait till the coding is finished. Does not mean to execute the test but to plan them.
- Involve the testers at an early stage, the requirements, the design.
- The SDLC model is not a box that you should try to fit your project into but rather a guide that should be costumized for your project so it has a good quality.

**What is the difference in testing between iterative and sequential:** Testing in sequential process models is performed late, in fixed phases, on completed artifacts, and each test level is usually executed only once, making defect detection more expensive and changes harder to implement. In contrast, iterative-incremental models integrate testing continuously within each iteration, with overlapping and repeated test levels, automated regression and integration tests, and evolving test suites. This approach allows early detection of defects, faster feedback, and better adaptability to changing requirements, as each increment is specified, developed, and validated in short cycles.

## Component Testing

- Component is the smallest building unit in a system and component testing is the first stage of testing after the actual code has been written to check that the module works correctly on its own before combining it with other modules.
- The word component itself means the smallest code that can be tested, and it differs from programming language to others, in OOP that is called a class.
- We shall not care about what the specific programming language calling it we will always call it component test.

### Component Test Objective:

- The main objective is to test whether this model implements the function correctly it tests against input to output. It tests both functional and non-functional requirements.
- Finding issues within the component
- We catch errors and fix it so its not passed to higher level systems
- Verify the functional and non-functional requirements, did we build the right thing for the user?
- Creates more quality and confidence in the specific component
- Check for qualities that can be very hard to test and fix lately, like maintainability and efficiency.

**Robustness Test:** is the ability of a module to handle wrong data, unexpected data gracefully without crashing the system. Components interact with neighboring components those components sometimes pass incorrect data or unexpected data, our component should be tested for such cases to handle these errors without crashing the system.

**Efficiency Test:** it is how economically a component uses system resources (CPU, memory, network, I/O). we shall measure the CPU, RAM consumption of the component to make sure it does not eat up the memory.

**Maintainability Test:** Maintainability is how easy its to understand, use, extend, debug, enhance the component that you already has. For good maintainability the code should be well documented, maintainability cannot be checked by DAST it should be done by peer reviews and SAST

**Component Test – Test Basis:** The component test basis consists of all artifacts describing the component, including detailed design, program code, data and behavior models, pre- and post-conditions, and invariants. In Agile or test-driven development, automated test cases themselves serve as both the specification and executable tests, defining expected behavior before or during implementation.

**Component Test – Test Objects:** Test Objects are the components that are going to be actually tested.

These modules can be, data structures, classes, database module, code, unit, module.

Each module is tested in an isolated environment, so any failure in the module is of that module.

A component may include multiple components inside it, test all of them but never test interactions with other modules outside this one.

### **Component Test – Test Environment:**

A test environment for a component offers everything a component needs to start the test since we isolate the module and a lot of the time the module needs data or to be called by another module so we need to prepare the environment.

This environment consist of 3 main things the test object ( the actual code that we will test), and the Driver; acts as an upper module that in real life scenario will call our component and provide some data for it, and the stub/dummy that module acts as its below the test object by providing dummy data for the component to continue to run.

A problem with the dummy or the mock is that sometimes you need to make those test cases for edge cases, you should include false, incorrect, unexpected input in the dummy.

Component testing is a development-related activity, often performed by developers (“developer test”), requiring access to code and interfaces to call the test object. Developers write, execute, and debug tests at this stage. In Agile/TDD (Test-Driven Development), automated component tests can be written before code implementation, ensuring early verification of expected behavior.

### **Component Test – Test Strategy:**

Component testing is a white-box testing activity where test cases are designed using knowledge of the internal structure and logic of the component. With program code available during execution, variables and internal states can be observed, allowing developers to easily identify correct or incorrect behavior and pinpoint defects directly within the component.

In practice, component testing is a hybrid of black-box (driven from req) and white-box testing: test cases are derived from requirements, while structural coverage is used as an end criterion. Large software systems often limit access to code or require testing of composite components, making full white-box testing impractical. Test focus—elementary or composite components—should be defined in integration and test planning to ensure efficient and effective testing.